

图论与搜索

DFS 深度优先搜索

用途：枚举、路径搜索、连通性。

常见坑：注意回溯还原（如标记数组清零），加剪枝防超时。

F. Parabola Independence 寻找最常路

- **核心模型：**寻找最长路，数学几何性质
- **修正逻辑 (Patch)：**把图分为上下两部分，按照规则建立DAG，通过dfs（好久没写好生疏）找到上下对于这个点而言最长路径，夹一下出结果
- **难点：**数学几何性质
- **关键代码：**

```
struct line
{
    int a, b, c;
};

bool canwalk(line a, line b)
{
    int chaa = a.a - b.a;
    int chab = a.b - b.b;
    int chac = a.c - b.c;
    if (a.a > b.a)
    {
        return chab * chab < 4 * chaa * chac;
    }
    else if (chaa == 0)
    {
        return chab == 0 && chac > 0;
    }
    else
    {
        return false;
    }
}

void solve()
{
    int n;
    cin >> n;
    vector<line> ls(n);
    vector<vi> mp(n);
    vector<vi> fmp(n);
    rep(i, 0, n - 1)
    {
        int aa, bb, cc;
```

```
        cin >> aa >> bb >> cc;
        ls[i].a = aa;
        ls[i].b = bb;
        ls[i].c = cc;
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (canwalk(ls[i], ls[j]))
            {
                mp[i].push_back(j);
                fmp[j].push_back(i);
            }
        }
    }

    vi dpu(n, 0);
    vi dpd(n, 0);

    auto dpp = [&](auto self, int a) -> int
    {
        if (dpu[a] != 0)
            return dpu[a];
        int cnt = 0;
        for (int i = 0; i < mp[a].size(); i++)
        {
            cnt = max(cnt, self(self, mp[a][i]));
        }
        return dpu[a] = cnt + 1;
    };

    auto dpq = [&](auto self, int a) -> int
    {
        if (dpd[a] != 0)
            return dpd[a];
        int cnt = 0;
        for (int i = 0; i < fmp[a].size(); i++)
        {
            cnt = max(cnt, self(self, fmp[a][i]));
        }
        return dpd[a] = cnt + 1;
    };

    for (int i = 0; i < n; i++)
    {
        int ans = dpp(dpp, i) + dpq(dpq, i) - 1;
        cout << ans << ' ';
    }
    cout << '\n';
}
```

流沙 (dfs找子树)

- **核心模型:** 贪心, 继承, dp
- **思维误区 (Bug):**
- **修正逻辑 (Patch):**
- **关键代码:**

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> nums[i];
    }

    vector<vi> mp(n + 1);
    for (int i = 0; i < n - 1; i++)
    {
        int u, v;

        cin >> u >> v;
        mp[u].push_back(v);
        mp[v].push_back(u);
    }

    vi dp(n + 1, 0);
    vi sum(n + 1, 0);
    vi tr(n + 1, 1);
    vi ans(n + 1);
    auto dfs = [&](int a, int fa, auto self) -> void
    {
        sum[a] = nums[a];

        int pans = INF;
        bool lv = 1;
        for (auto it : mp[a])
        {
            if (it == fa)
            {
                continue;
            }
            lv = 0;
            self(it, a, self);

            sum[a] += sum[it];
            tr[a] += tr[it];
            pans = min(pans, ans[it]);
        }
        if (lv)
```

```

        ans[a] = nums[a];
    else
        ans[a] = min(pans, sum[a] / tr[a]);
    };

    dfs(1, -1, dfs);
    for (int i = 1; i <= n; i++)
    {
        cout << ans[i] << ' ';
    }
    cout << '\n';
}

```

分治->Generate01String

- **核心模型**: 括号匹配, 单调性分治递归搜索树
- **思维误区 (Bug)**: 第一时间没有观察到其树状递归单调性
- **修正逻辑 (Patch)**: 注意递归顺序最大的出发点把-1和n看作一个虚空大括号, 每加上一个小括号子树进去就要加一次
- **关键代码**:

```

void solve()
{
    string s;
    cin >> s;

    if (2 * count(all(s), '0') != s.size())
    {
        cout << -1 << '\n';
        return;
    }

    vector<pii> stk;
    vector<pii> bt;
    cout << count(all(s), '0') << '\n';
    for (int i = 0; i < s.size(); i++)
    {
        if (stk.empty() || s[i] == stk.back().first)
        {
            stk.push_back({s[i], i});
        }
        else
        {
            bt.push_back({stk.back().second, i});
            stk.pop_back();
        }
    }
    sort(all(bt), [](pii a, pii b)
        {

```

```

        if(a.first!=b.first) return a.first<b.first;
        else a.second>b.second; });
int tp = 0;
int pos = 1;
vector<pii> ans;
auto dfs = [&](auto &&self, int L, int R) -> void
{
    if (L >= R)
        return;

    if (L != -1)
    {
        if (s[L] == '0')
            ans.push_back({pos, 1});
        else
            ans.push_back({pos, 2});
    }
    while (tp < bt.size() && bt[tp].second < R)
    {
        int a = bt[tp].first;
        int b = bt[tp].second;
        tp++;
        self(self, a, b);
        pos++;
    }
};
dfs(dfs, -1, s.size());

for (auto [a, b] : ans)
{
    cout << a << ' ' << b << "\n";
}
}

```

小猫爬山

```

int n, w, ans = 20;
vector<int> lcs(20);
void dfs(int stepnum, int lcnun, const vector<int> &cats) {
    if (lcnun >= ans) return;
    if (stepnum == n) {
        ans = lcnun;
        return;
    }
    for (int j = 1; j <= lcnun; j++)
        if (lcs[j] + cats[stepnum] <= w) {
            lcs[j] += cats[stepnum];
            dfs(stepnum + 1, lcnun, cats);
            lcs[j] -= cats[stepnum];
        }
}

```

```

    lcs[lcnum + 1] = cats[stepnum];
    dfs(stepnum + 1, lcnum + 1, cats);
    lcs[lcnum + 1] = 0;
}
void solve() {
    cin >> n >> w;
    vector<int> cats(n + 1);
    for (int i = 0; i < n; i++) cin >> cats[i];
    sort(cats.rbegin(), cats.rend());
    dfs(0, 1, cats);
    cout << ans << '\n';
}

```

复杂度： $O(2^n)$ 最坏，剪枝后更优。

E2. Interactive Graph (Hard Version)

- **核心模型：**字典序 + 计数：核心的跳跃量 $c[u]$ 本质是在字典序排列里做区间跳跃，跟数位DP的思想非常像
- **思维误区 (Bug):**
- **修正逻辑 (Patch):**
- **关键代码:**

BFS

P1434 [SHOI2002] 滑雪

- **链接:** [题目链接](#)
- **算法类型:** 拓扑dp
- **此为标准代码请认真研读**
- **AC 代码:**

```

vi dr = {1, 0, -1, 0};
vi dc = {0, 1, 0, -1};
void solve()
{
    vvi height(101, vi(110));
    vvi indigree(101, vi(110, 0));
    int c, r;
    cin >> r >> c;
    for (int i = 1; i <= r; i++) // 我还是喜欢1 based存图
    {
        for (int j = 1; j <= c; j++)
        {
            cin >> height[i][j];

```

```

    }
}
for (int i = 1; i <= r; i++)
{
    for (int j = 1; j <= c; j++)
    {
        for (int k = 0; k < 4; k++)
        {
            if (i + dr[k] <= 0 || i + dr[k] > r || j + dc[k] <= 0 || j + dc[k]
> c)
                continue;
            if (height[i + dr[k]][j + dc[k]] < height[i][j])
            {
                indigree[i][j]++;
            }
        }
    }
}
int ans=0;
vvi dp(101,vi(101,1));
queue<pair<int, int>> pos;
for (int i = 1; i <= r; i++)
    for (int j = 1; j <= c; j++)
        if (indigree[i][j] == 0)
            pos.emplace(i, j);
while (!pos.empty())
{
    auto [x, y] = pos.front();
    pos.pop();
    for (int k = 0; k < 4; k++)
    {
        if (x + dr[k] <= 0 || x + dr[k] > r || y + dc[k] <= 0 || y + dc[k] >
c)
            continue;
        if (height[x + dr[k]][y + dc[k]] > height[x][y])
        {
            dp[x + dr[k]][y + dc[k]]=max(dp[x][y]+1,dp[x + dr[k]][y + dc[k]]);
            ans=max(ans,dp[x + dr[k]][y + dc[k]]);
            indigree[x + dr[k]][y + dc[k]]--;
            if(indigree[x + dr[k]][y + dc[k]]==0)
                pos.emplace(x + dr[k],y + dc[k]);
        }
    }
}
if(!ans)cout<<"1";
else cout<<ans;
}

```

- 思路:

- 1. 建图 (这里用反图更方便)
- 2. 计算入度

- 3.\ 入度为0的点入队
- 4.\ BFS:
- 5.\ 答案 = max(dp[])

```
//BFS
while (!q.empty()) {
    u = 出队
    for each v that u → v (原图: u 可以到达 v)
        dp[v] = max(dp[v], dp[u] + 1)
        inDegree[v]--
        if (inDegree[v] == 0) 入队
    }
}
``````

BFS例题: P1443 马的遍历

- **题号**: P1443
- **链接**: [题目链接]<https://www.luogu.com.cn/problem/P1443>
- **算法类型**: BFS
- **AC 代码**:

```cpp
nn[x][y]=0;
queue<pair<int, int>> bfs; // 建立一个以点坐标为项目的队列
bfs.emplace(x, y);
while (!bfs.empty())
{
    auto [h, w] = bfs.front();
    bfs.pop();
    for (int i = 0; i < 8; i++)
    {
        int hsh = h + dx[i];
        int ljl = w + dy[i];
        if (hsh < 1 || hsh > n || ljl < 1 || ljl > m || !nn[hsh][ljl])
            continue;
        nn[hsh][ljl] = 0;
        mp[hsh][ljl] = mp[h][w] + 1;
        bfs.emplace(hsh, ljl);
    }
}
}
```

- **注意事项:**
 - 记得BFS不要漏了bool visit数组
 - 记得BFS状态转移 (?) 方程 $mp[hsh][ljl] = mp[h][w] + 1;$
 - 最短路问题 不是“另一种算法”，它就是 DP 在无权图上的高效实现
- **改进思路:**
 - 考虑严格 $cnt == k$ 的情况，调整 check 函数。

P1825 [USACO11OPEN] Corn Maze S

- **题号:** P1825
- **链接:** [题目链接](#)
- **算法类型:** BFS
- **记录原因:**
 - AC了, 终点看引参数进数组! 不要天天被神秘UB卡住脑子
 - 这道题无非是标准BFS上面加了个规则函数: 这道题放这里就是教你如何写有规则的搜索
- **AC 代码:**

```
void goincsm(const vector<vector<int>> &mp, int &xx, int &yy)
{
    rep(i, 1, n)
    {
        rep(j, 1, m)
        {
            if (mp[i][j] == mp[xx][yy] && (i != xx || j != yy))
            {
                xx = i;
                yy = j;
                return;
            }
        }
    }
}
```

- **注意事项:**
 - 取地址代表可更改 `int &xx`: 适用于那些总是要更改的量
 - `const vector<vector<int>> &mp` 既安全又高校的传图方式

P2895 [USACO08FEB] Meteor Shower S

- **题号:** P2895
- **链接:** [题目链接](#)
- **算法类型:** BFS
- **错误原因:**
 - 边界检查边界检查边界检查
 - 时间更新逻辑
 - vis数组限制逻辑 (依旧时间更新逻辑)
- **AC 代码:**

```
vi dx = {-1, 0, 1, 0};
vi dy = {0, 1, 0, -1};

struct flashlight
{
    int xi, yi, ti;
};

void solve()
```

```

{
    int m;
    cin >> m;
    vector<vector<bool>> vis(305, vector<bool>(305, 1));
    vector<vector<bool>> hited(305, vector<bool>(305, 1));
    vector<vector<int>> mp(305, vector<int>(305, INF));
    vector<flashlight> bar(m);
    vector<flashlight> safe;
    rep(i, 0, m - 1)
    {
        cin >> bar[i].xi >> bar[i].yi >> bar[i].ti;
        hited[bar[i].xi][bar[i].yi] = 0;
        rep(j, 0, 3)
        {
            int xd = bar[i].xi + dx[j];
            int yd = bar[i].yi + dy[j];
            if (xd < 0 || xd > 302 || yd < 0 || yd > 302)
                continue;
            hited[xd][yd] = 0;
        }
    }
    sort(bar.begin(), bar.end(), [](flashlight a, flashlight b)
        { return a.ti < b.ti; });
    rep(i, 0, 304)
    {
        rep(j, 0, 304)
        {
            if (hited[i][j] == 1)
                safe.push_back({i, j, 0});
        }
    }

    queue<pii> pos;
    pos.emplace(0, 0);
    vis[0][0] = 0;
    int time = 0;
    while (!pos.empty())
    {
        int sz = SZ(pos); // 当前层有多少个点
        auto it = find_if(all(bar), [time](const flashlight &a)
            { return a.ti == time+1; });
        while (it != bar.end())
        {
            int xo = it->xi;
            int yo = it->yi;
            vis[xo][yo] = 0;
            rep(i, 0, 3)
            {
                int xoo = xo + dx[i];
                int yoo = yo + dy[i];

                if (xoo < 0 || xoo > 302 || yoo < 0 || yoo > 302 || vis[xoo][yoo]
== 0)
                    continue;

```

```

        vis[xoo][yoo] = 0;
    }
    it = std::find_if(++it, bar.end(),
                     [time](const flashlight &a)
                     { return a.ti == time+1; });
}
rep(layer, 0, sz - 1)
{
    auto [x, y] = pos.front();
    pos.pop();

    rep(i, 0, 3)
    {
        int xx = x + dx[i];
        int yy = y + dy[i];
        if (xx < 0 || xx > 302 || yy < 0 || yy > 302 || vis[xx][yy] == 0)
            continue;

        vis[xx][yy] = 0;
        mp[xx][yy] = time + 1;
        pos.emplace(xx, yy);
    }
}
time++;
}
int ans = INF;
rep(i, 0, SZ(safe) - 1)
{
    ans = min(ans, mp[safe[i].xi][safe[i].yi]);
}
if (ans == INF)
{
    cout << "-1";
    return;
}

cout << ans;
}

```

- **注意事项:**

- 注意：这道题没有规定地图大小，对于流星影响（限制点）最大可到301*301.所以我们的搜索应该开到303（最保险），包括continue地搜索限制
- 注意time更新逻辑：lily只能在ti之前到达这个点，所以对于每个影响的点需要参考的是time+1;
- 注意bfs每层地更新逻辑：每次入队都是time (i) 时间点可到地所有点

- **结构体lambda搜索技巧:**

根据ti寻找bar中所需要地项，返回迭代器

```
auto it = find_if(all(bar), [time](const flashlight &a)
                 { return a.ti == time+1; });
```

直接从迭代器里面提取出来我们要的：->

```

int xo = it->xi;
int yo = it->yi;
如果符合条件的项目存在
while (it != bar.end())
寻找下一个迭代器
it = std::find_if(++it, bar.end(),
                  [time](const flashlight &a)
                  { return a.ti == time+1; });

```

P1162 填涂颜色 提供深搜广搜两种做法

- 题号: P1162
- 链接: [题目链接](#)
- 算法类型: 搜索板子
- 错误原因:
 - 注意从四周寻找'0'切入
- AC 代码广搜版:

```

vector<vector<int>> mp(35, vi(35));
vector<vector<bool>> vis(35, vector<bool>(35, 1));
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= n; j++)
    {
        cin >> mp[i][j];
    }
}
queue<pair<int, int>> bfs;
//bfs.emplace(1, 1);
for (int i = 1; i <= n; i++)
{
    if (mp[1][i]==0) bfs.emplace(1, i);
    if (mp[i][1]==0) bfs.emplace(i, 1);
    if (mp[n][i]==0) bfs.emplace(n, i);
    if (mp[i][n]==0) bfs.emplace(i, n);
}
while (!bfs.empty())
{
    auto [lg, cf] = bfs.front();
    bfs.pop();
    for (int i = 0; i < 4; i++)
    {
        int xlg = lg + dx[i];
        int xcf = cf + dy[i];
        if (xlg < 1 || xlg > n || xcf < 1 || xcf > n || !vis[xlg][xcf] ||
mp[xlg][xcf] == 1)
        {
            continue;
        }
    }
}

```

```

        vis[xlg][xcf] = 0;
        mp[xlg][xcf] = -1;
        bfs.emplace(xlg, xcf);
    }
}

```

- **AC 代码深搜版:**

```

void dfs(int a, int b, vector<vector<int>> &mp)
{
    if (mp[a][b] != 0)
        return;
    if (mp[a][b] == 0)
        mp[a][b] = -1;
    for (int i = 0; i < 4; i++)
    {
        int xlg = a + dx[i];
        int xcf = b + dy[i];
        //dbg(xlg,xcf);
        if (xlg < 1 || xlg > n || xcf < 1 || xcf > n)
            continue;
        dfs(a + dx[i], b + dy[i], mp);
    }
    return;
}
//solve里面的
for (int i = 1; i <= n; i++)
{
    dbg(i);
    dfs(i, 1, mp);
    dfs(1, i, mp);
    dfs(n, i, mp);
    dfs(i, n, mp);
}

```

- **注意事项:**
 - 注意从四周引入点
 - 注意入队时机
- **改进思路:**
 - 学习两种搜索方式

无向图分组

用途: 求连通分量数。

常见坑: 双向边需存两次，初始化标记数组。

```

vector<vector<int>> a(2e5 + 10);
vector<int> kk(2e5 + 10);

```

```

void dfs(int k) {
    for (int i = 0; i < a[k].size(); i++)
        if (kk[a[k][i]] == 0) {
            kk[a[k][i]] = 1;
            dfs(a[k][i]);
        }
}

void solve() {
    int n, m, ans = 0;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int p, q;
        cin >> p >> q;
        a[p].push_back(q);
        a[q].push_back(p);
    }
    for (int j = 1; j <= n; j++)
        if (kk[j] == 0) {
            kk[j] = 1;
            dfs(j);
            ans++;
        }
    cout << ans << '\n';
}

```

复杂度： $O(n+m)$ ，空间 $O(n+m)$ 。

图论：常见做法：反向建边

P4017 最大食物链计数（拓扑排序）

- **题目概述** 给出一张有向无环图，求出最长路径的数量（最长路径定义：入度为0的点到初读为0的点）， n 是节点数量， m 是路径数量
- **数据范围**: $n:2e3, m:1e5$
- **初始思路**；记忆化加DFS
- **正解思路**：DFS+DP
- **AC代码**

```

int mod = 80112002;
int n, m;
vi mem(5005, -1);
int dfs(int u, const vector<vi> &dw, const vi&in, const vi&out)
{
    if (mem[u] != -1)
    {
        return mem[u];
    }
    if (in[u] == 0)
    {
        mem[u] = 1;
    }
}

```

```

        return 1;
    }
    mem[u]=0;
    for (int v : dw[u])
    {
        // mem[u] = max(mem[u], dfs(v, dw,in,out) % mod + 1);
        mem[u]=(dfs(v,dw,in,out)+mem[u])%mod;
    }
    return mem[u];
}

void solve()
{
    vector<vi> dw(5005);
    cin >> n >> m;
    vi out(n + 1);
    vi in(n + 1);
    rep(i, 0, m - 1)
    {
        int d, f;
        cin >> d >> f;
        dw[f].push_back(d);
        out[d]++; //需要计算入度和出度是为了辨别什么时候开始什么时候结束
        in[f]++;
    }
    int ans = 0;
    for (int i = 1; i <= n; i++)
    {
        if (out[i] == 0)
            // ans=max(ans,dfs(i,dw)%mod);
            ans = (ans + dfs(i, dw,in,out)) % mod;
    }
    cout << ans << '\n';
}

```

• AC代码:拓扑排序

```

void solve()
{
    vector<vi> dw(5005);
    cin >> n >> m;
    vi out(n + 1);
    vi in(n + 1);
    rep(i, 0, m - 1)
    {
        int d, f;
        cin >> d >> f;
        dw[f].push_back(d);
        out[f]++;
        in[d]++;
    }
    int ans = 0;

```

```

queue<int> dl;
vi lx(n + 1,0);
for (int i = 1; i <= n; i++)
{
    if (in[i] == 0)
    {
        dl.emplace(i);
        lx[i]=1;
    }
}

while (!dl.empty())
{
    auto hsh = dl.front();
    dl.pop();

    for (auto hsh2 : dw[hsh])
    {
        lx[hsh2]=(lx[hsh2]+lx[hsh])%mod;
        in[hsh2]--;
        if(in[hsh2]==0)
        {
            dl.emplace(hsh2);
        }
    }
}
for(int i=1;i<=n;i++)
{
    if(!out[i])
    {
        ans=(ans+lx[i])%mod;
    }
}
cout << ans << '\n';
}

```

- 拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。
- 作用：
 - 确定任务执行顺序
 - DAG 上的动态规划
 - 检测环路:如果拓扑排序无法将所有节点都加入到最终的序列中（
 - **"顺序"、"依赖"、"先决条件"，或者需要在一个有向图中进行基于依赖的计算（如 DP）时

图论：dijkstra

简单 Dijkstra 模板题

P4779 【模板】单源最短路径（标准版）

- 题号: P4799
- 链接: [题目链接](#)

- **算法类型:** 图论模板
- **AC 代码:**

```
void solve()
{
    ll n,m,s;
    cin>>n>>m>>s;
    //前面是点,后面是距离
    vector<vector<pair<int,ll>>> mp(n+1);
    rep(i,0,m-1)
    {
        int u,v,w;
        cin>>u>>v>>w;
        mp[u].push_back({v,w});
        // mp[v].push_back({u,w});这道题是有向边所以不要入两次!!
    }

    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>pos ;
    vector<ll> dist(n+1,LINF);
    dist[s]=0;
    pos.emplace(0,s);

    while(!pos.empty())
    {
        auto [nowd,nowp] = pos.top();
        pos.pop();
        if(nowd>dist[nowp])continue;
        for(auto [nextp,nextd]:mp[nowp])
        {
            if(dist[nextp]>dist[nowp]+nextd)
            {
                dist[nextp]=dist[nowp]+nextd;
                pos.emplace(dist[nextp],nextp);
            }
        }
    }
    rep(i,1,n)
    {
        cout<<dist[i]<<' ';
    }
}
```

- **注意事项:**
 - 注意Dijkstra算法用最小堆优化可以时间复杂度最低
 - Dijkstra算法只能处理非负权路径问题
 - 为什么不用队列? : 贪心最快, 如果是菊花图复杂度会退化到nm
 - 含负权路用什么算法? 用队列
- **思路:**
 - 认真研读并学习114514次

水群

- 题号: D
- 链接: [题目链接](#)
- 算法类型: Disjkstr最短路
- 错误原因:
 - 看成DP了!! 有向无环有向无环有向无环!
 - 迪克算法还在追我
 - 最短路问题
- AC 代码:

```
void solve()
{
    vector<vector<pair<ll, int>>> mp(n + 1);
    for (int i = n-1; i >= 1; i --)
    {
        mp[i].push_back({x, i + 1});
        mp[i + 1].push_back({x, i});
    }
    rep(i, 0, m - 1)
    {
        int a, b;
        cin >> a >> b;
        mp[a].push_back({y, b});
        // mp[b].push_back({y, a});
    }
    【迪克算法】
    cout<<dist[1]<<'\n';
}
```

- 思路:
 - 没什么好说的, 迪克算法模板, 我都懒得贴出代码, 详情请见[简单 Dijkstra 模板题] <#p4779-模板单源最短路径标准版>

代号N

- 题号: E
- 链接: [题目链接](#)
- 算法类型: 带权无向图[相似题目学习](#)

精简题干: 定义一棵树:共n个节点:一个度数为3的节点,若干个度数为2的节点,3个度数为1的节点.给出n-1条边,给出两个端点以及边权,可以操作k次将某条边边权变成0.求根节点到叶节点的度数最大值的最小值.

- AC 代码:

```
struct edge
{
    int to, w;
```

```

};
int cmp(const vi &sums)
{
    if(sums[0]>sums[1])
    {
        if(sums[0]>sums[2])
        {
            return 0;
        }
        else return 2;
    }
    else
    {
        if(sums[1]>sums[2])
        {
            return 1;
        }
        else return 2;
    }
}

void solve()
{
    int n, k;
    cin >> n >> k;
    vector<vector<edge>> mp(n + 1);
    vi nums(n + 1);
    for (int i = 0; i < n-1; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        mp[u].push_back({v, w});
        mp[v].push_back({u, w});
        nums[u]++;
        nums[v]++;
    }
    auto it = find(all(nums), 3);
    int sdot = (it - nums.begin());
    vector<vector<int>> chain(3);
    vi sums(3);
    vector<priority_queue<int>> pq(3); // <--- 改动: 为每条链维护大根堆
    for (int i = 0; i < 3; i++)
    {
        queue<pii> pos;
        pos.emplace(mp[sdot][i].to, sdot);
        chain[i].push_back(mp[sdot][i].w);
        sums[i] += mp[sdot][i].w;
        pq[i].push(mp[sdot][i].w); // <--- 改动: 把边权加入堆

        while (!pos.empty())
        {
            auto [now, par] = pos.front();
            pos.pop();

```

```

        for (auto &nextdot : mp[now])
        {
            if (nextdot.to == par)
                continue;
            else
            {
                chain[i].push_back(nextdot.w);
                sums[i] += nextdot.w;
                pq[i].push(nextdot.w);          // <--- 改动: 把遍历到的边权
            }
            pos.emplace(nextdot.to, now);
        }
    }
}
while(k--)
{
    int idx = cmp(sums);
    if (pq[idx].empty()) break;              // 防止空链
    int mx = pq[idx].top();
    pq[idx].pop();                          // O(log len)
    sums[idx] -= mx;
    // 下面的两行可以删掉, 保留只为保持原结构
    // sums[cmp(sums)] -= *max_element(all(chain[cmp(sums)]));
    // chain[cmp(sums)].erase(max_element(all(chain[cmp(sums)])));
}
cout << sums[cmp(sums)];
}

```

加入堆

• 思路剖析:

- 非常简单的大数据和和的最大值的最小值.解法一:sort之后逐个pop_back()(注意vector的pop是最后一位,所以要从小到大排序).解法二:维护最大根堆priority_queue<int>,n次操作复杂度nlogn.
- 事实上解法一是优解,但是为了学习这个有意思的容器这道题用的做法是最大根堆
- 这道题的难点是读懂题...然后是建带权无向图...依旧模板.
- **记得**带权无向图两个端点都pushback一次
- 然后就是用bfs带着上一个节点和下一个节点广度优先探索(这个还能用dfs解法,下文附上,作为图论路径转移学习,请严肃学习114514次)

大部分图论用BFS做,在面临剪枝需求/需要回溯/所有方案的时候用DFS做

• 细节注意:

- 二维拓展数组记得这样开vector<vector<edge>> mp(n + 1);不会爆空间
- Cpp17不支持结构体取地址,就这么写auto [now, par] = pos.front();
- 记得k次操作不一定用玩一定要提前出循环if (pq[idx].empty()) break;以免UB

• 新结构

- priority_queue: 自动维护堆的“插入+弹出最大/最小”工具,“贪心/最短路/Top-K/滑动窗口最大值”都能靠它快速实现
- 注意优先队列没办法删除除了堆顶以外的元素,所以**注意**if (d > dist[u]) continue;

分层图

逃出生天

- **核心模型**:一句话概括题意/数学本质 (如: 中位数贪心 / 差分约束)
- **思维误区 (Bug)**:记录第一直觉为什么错了 (如: 以为是DP其实是贪心 / 读错题)
- **修正逻辑 (Patch)**:下次看到什么特征, 要修正为正确思路
- **关键代码**:

```
vi dx = {1, 0, -1, 0, 0};
vi dy = {0, 1, 0, -1, 0};
int n, m;
int laohupos(int stpos, int fx, int t)
{
    int idx;

    if (fx == 1)
    {
        idx = stpos - 1;
    }
    else
    {
        idx = 2 * m - 2 - (stpos - 1);
    }

    return (idx + t) % (2 * m - 2);
}
struct hhh
{
    int x, y, t;
};
void solve()
{
    cin >> n >> m;
    vector<vector<vi>> hsh(n + 1, vector<vi>(m + 1, vi(2 * m - 2, 0)));
    vector<pii> tiger;
    for (int i = 1; i <= n; i++)
    {
        int d, dd;
        cin >> d;
        char c;
        cin >> c;
        if (c == 'R')
            dd = 1;
        else
            dd = -1;
        tiger.push_back({d, dd});
    }
    vi laohu;
    for (int i = 1; i <= m; i++)
        laohu.push_back(i);
```

```

    for (int i = m - 1; i > 1; i--)
        laohu.push_back(i);

    queue<hhh> pos;
    pos.push({1, m, 0});
    hsh[1][m][0] = 1;
    while (!pos.empty())
    {
        auto [nx, ny, nt] = pos.front();
        pos.pop();

        if (nx == n && ny == 1)
        {
            cout << "Yes" << '\n';
            return;
        }
        int tm = nt + 1;
        for (int i = 0; i < 5; i++)
        {
            int xx = nx + dx[i];
            int yy = ny + dy[i];

            if (xx < 1 || xx > n || yy < 1 || yy > m)
                continue;
            if (hsh[xx][yy][tm%(2*m-2)])
                continue;

            if(laohu[laohupos(tiger[xx-1].first,tiger[xx-1].second,nt)]==yy)continue;
            if(laohu[laohupos(tiger[xx-1].first,tiger[xx-1].second,tm)]==yy)continue;

            hsh[xx][yy][tm%(2*m-2)]=1;
            pos.push({xx,yy,tm%(2*m-2)});
        }
    }
    cout << "No" << endl;
}

```

G_Exploration

- **核心模型**: 分层图dp
- **思维误区 (Bug)**: 一开始看见权值/2地往下递减错误的想用bfs。bfs本质上就是一点一点枚举路径，复杂度是指数级别的
- **修正逻辑 (Patch)**: $ce[v][k]$ 是指：从v出发走k步最大地值。题目是要求/边权，我们可以反向思考成从某点出发*边权。可以证明出单调性
- 发现重复子问题 → 想到DP
- 发现步数上界只有30 → 把步数作为DP的维度
- 发现正向状态太多 → 翻转问题方向
- **关键代码**:

```
void solve()
{
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<pii>> mp(n + 1);
    for (int i = 0; i < m; i++)
    {
        int u, v, d;
        cin >> u >> v >> d;
        mp[v].push_back(make_pair(u, d));
    }
    vector<vector<int>> dp(40, vi(n + 1, 0));
    for (int i = 1; i <= n; i++)
    {
        sort(mp[i].begin(), mp[i].end(), [](pii a, pii b)
            { return a.second < b.second; });
    }
    for (int i = 0; i < n + 1; i++)
    {
        dp[0][i] = 1;
    }
    for (int k = 1; k <= 30; k++)
        for (int i = 1; i <= n; i++)
        {
            for (auto it : mp[i])
            {
                int nxt = it.first;
                int gap = it.second;

                dp[k][nxt] = max(dp[k][nxt], min(dp[k - 1][i] * gap, (int)2e9));
            }
        }

    while (q--)
    {
        int po, val;
        cin >> po >> val;
        int ans = 0;
        for (int i = 0; i <= 30; i++)
        {
            if (dp[i][po] > val)
            {
                ans = i;
                break;
            }
        }
        cout << ans << '\n';
    }
}
```

tarjan

图的遍历 (tarjan算法)

- **题目描述**: 给出一张有向图 (不保证无环), 节点编号1到n, 求每个节点能到达的最大编号
- **错误解法: 我一开始想到的**: dfs加记忆化, 从入度为0的点开始dfs到出度为0的点, 每个点的答案在确认的时候和自己还有其他的路径比较一下;
 - **错解中的逻辑问题**
 - 只D入度为0的点, 但是当图中出现环就会漏掉
 - 为了剪枝设计了vis数组来标记有没有被访问过, 但是没有重置: 这个题目不保证无环, 所以必须重置, 甚至说这个vis的存在就没啥必要
 - 在有环图中, 当 DFS 访问到一个正在递归栈中的点时, 说明遇到了环。正确的 DFS 应该使用三态
 - 在ans中要先初始化ans[i]=i, 因为每个点都可以到达自己。 **以免出现没有连通的点**

vis在以下情况不需要重置: 当你的图是有向无环图 (DAG) 或者你对每个点的计算结果是确定的、最终的、且不会随起点变化时, 你可以设置 vis 后不再变回 1 当他作为强连通分量 (SCC) 标记也可以不重置 比如说在这道题里面, 当你反向建图让他从最大数字的点开始DFS的时候vis就可以选择不重置: **因为N在这条路径上是最大的点, 后面的状态可以通过继承前面的状态来确定答案**

- **正解一: 反向建图**: 原理: 贪心保证剪枝成功(一旦一个点的答案 $A(v)$ 被确定, 它就是正确的最大值, 且永远不需要重新计算。)
 - 问题: 为什么 Visited(v) 可以永不重置?
 - 思考: 当我们从 i 开始 DFS 时, 如果 v 还没有 Visited, 我们就设 $A(v) = i$ 。在此之前, 所有的 $k > i$ 都没有在 G' 中到达 v (否则 v 早就被标记了)。因此, 没有比 i 更大的点是 v 可达的。
- **正解二: tarjan算法**: 求“缩点”操作的高效算法
 - 缩点然后顺序dfs+dp。求完强连通分量可以保证图片是DAG
- **检查自己方案合理性的思考路径**:
 - 1.\ 图的特性: 有向? 无环? 连通性? (有没有孤立的点) 边权? (负权边?)
 - 2.\ 做法检验: 有没有环? 依赖顺序是什么? 状态是否能持久? (影响剪枝) (在 DAG 上, 从拓扑序逆序 (即从终点开始) 计算是可靠的。原图 DFS 依赖于子问题的答案, 必须确保子问题先被计算。)
- **AC正解1(反向图思路)**

```
void dfs(int a, int f, const vector<vi> &mp, vi &ans)
{
    ans[a] = f;
    if (mp[a].empty())
        return;
    for (int hsh : mp[a])
    {
        if (ans[hsh] != -1)
        {
```



```

        continue;
    }
    dfs(hsh, f, mp, ans);
}
return;
}

void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vi> mp(n + 1);
    rep(i, 1, m)
    {
        int u, v;
        cin >> u >> v;
        mp[v].push_back(u);
    }
    vi ans(n + 1, -1);
    for(int i=n;i>=1;i--)
    {
        if(ans[i]!=-1)
        {
            continue;
        }

        dfs(i,i,mp,ans);
    }
    rep(i,1,n)
    {
        cout<<ans[i]<<' ';
    }
}

```

- **AC正解2**(Tarjan思路)